MCA Semester 1	Advanced Data Structure Lab
Name: Nitin Salunkhe	Title: Max Heap
Roll no: MCA2542	Date: 22.12.2025

Source Code:

```
import java.util.*;


class MaxHeap{
        int[] arr;//array to store heap elements
        int cnt; //no.of elements in heap
        int n; //Maximum capacity of heap

        public MaxHeap(int capacity){
                n=capacity;
                arr=new int[n];
                cnt=0;
        }
        //Parent
        public int parent(int i){
                return (i-1)/2;
        }
        //Left Child
        public int leftChild(int i){
                return (2*i)+1;
        }
        //Right Child
        public int rightChild(int i){
                return (2*i)+2;
        }
        //Swap
```

```
public void swap(int i, int j){
        int tmp=arr[i];
        arr[i]=arr[j];
        arr[j]=tmp;
}
//Insert
public void Insert(int x){
        //Heap full
        if(cnt==n){

                System.out.println("Heap is Full");
                return;
        }
        //insert x at the end of the array - to maintain almost complete property
        arr[cnt]=x;
        int curr=cnt; // Track the parent indices
        cnt++;

        //Reheap up - maintain order property
        while(curr>0 && arr[curr]>arr[parent(curr)]){
                swap(curr,parent(curr));
                curr=parent(curr);
        }
}

//Delete
public void Del(){
        if(cnt==0){
                System.out.println("Heap is Empty");
```

```
                return;
        }
        int max=arr[0];
        arr[0]=arr[cnt-1];
        cnt--;
        reheapDown(0);
        System.out.println("Element Removed is: "+max);
}
//reheapDown
public void reheapDown(int i){
        int largest = i;
        int left = leftChild(i);
        int right = rightChild(i);

        //find the largest among root, left and right child
        if(left<cnt && arr[left]>arr[largest]){
                largest=left;
        }
        if(right<cnt&&arr[right]>arr[largest]){
                largest=right;
        }
        //If the largest is not the root, swap and continue reheapDown
        if(largest!=i){
                swap(i,largest);
                reheapDown(largest);
        }
}
//Display
public void Display(){
```

```
            if(cnt==0){
                System.out.println("Heap is Empty!");
                return;
            }

            System.out.print("Heap Contains: ");
            for(int i=0;i<cnt;i++){
                System.out.print(arr[i]+" ");
            }
            System.out.println();
        }

}//end of MaxHeap
public class Heap{
    public static void main(String[] args){

            Scanner sc=new Scanner(System.in);
            MaxHeap h=new MaxHeap(10);
            int ch;

            do{
                System.out.println("Max Heap\n");

                System.out.println("1: Insert in the Heap");
                System.out.println("2: Delete From the Heap");
                System.out.println("3: Display the Heap");
                System.out.println("4: Exit\n");

                System.out.println("Enter Your Choice");
```

```
                ch=sc.nextInt();


                switch(ch){
                        case 1:
                                System.out.println("Enter A value: ");
                                int x=sc.nextInt();
                                h.Insert(x);
                                break;
                        case 2:
                                h.Del();
                                break;
                        case 3:
                                h.Display();
                                break;
                        case 4:
                                System.out.println("Exited!");
                                break;
                        default:
                                System.out.println("Incorrect Choice!");
                                break;


                }
        }while(ch!=4);
    }
}
```

Output:

```
C:\Users\mca2542\Desktop\DSA2542>java Heap
Max Heap

1: Insert in the Heap
2: Delete From the Heap
3: Display the Heap
4: Exit

Enter Your Choice
1
Enter A value:
45
```

```
Max Heap

1: Insert in the Heap
2: Delete From the Heap
3: Display the Heap
4: Exit

Enter Your Choice
3
Heap Contains: 90 65 87 45 11 13 22 16
```

```
Max Heap

1: Insert in the Heap
2: Delete From the Heap
3: Display the Heap
4: Exit

Enter Your Choice
2
Element Removed is: 90
```